

# Assignment 3: OpenGL & Phong Shading

作者：王洵

## 1 实验原理

### 1.1 Phong 局部光照与 Blinn-Torrance 高光

局部光照模型 (local illumination model) 将表面上一点的颜色分解为环境光项与各光源直接照明之和。设表面单位法线为  $\mathbf{N}$ ，指向光源的单位方向为  $\mathbf{L}$ ，指向观察者的单位方向为  $\mathbf{V}$ ，漫反射反照率为  $\mathbf{k}_d$ ，高光反照率为  $\mathbf{k}_s$ ，光源颜色为  $\mathbf{c}_{\text{light}}$ 。Lambert 漫反射强度正比于  $\max(\mathbf{N} \cdot \mathbf{L}, 0)$ ；当  $\mathbf{N} \cdot \mathbf{L} \leq 0$  时，入射光无法到达切平面，该光源漫反射贡献为零。

Blinn-Torrance 高光以半角向量

$$\mathbf{H} = \text{normalize}(\mathbf{L} + \mathbf{V})$$

代替传统 Phong 模型中的反射方向，高光强度取  $(\mathbf{N} \cdot \mathbf{H})^n$ 。指数  $n$  控制高光瓣宽度： $n$  越大瓣越窄，高光越尖锐。单光源下的直接照明为

$$\mathbf{c}_{\text{local}} = \max(\mathbf{N} \cdot \mathbf{L}, 0) \mathbf{c}_{\text{light}} \odot \mathbf{k}_d + \max(\mathbf{N} \cdot \mathbf{H}, 0)^n \mathbf{c}_{\text{light}} \odot \mathbf{k}_s,$$

其中  $\odot$  为逐分量相乘。方向光源位于无穷远，场景内入射方向恒定，距离平方反比衰减可忽略。像素颜色在环境光  $\mathbf{c}_{\text{ambient}} \odot \mathbf{k}_d$  之上累加各光源的  $\mathbf{c}_{\text{local}}$ 。

### 1.2 掠射角高光伪影

当高光指数  $n$  较小时，高光瓣较宽。在掠射角附近  $\mathbf{N} \cdot \mathbf{L}$  接近零但仍为正， $(\mathbf{N} \cdot \mathbf{H})^n$  仍可能较大，从而在几何上接近背光的一侧出现不合理亮斑。将高光项再乘以  $\mathbf{N} \cdot \mathbf{L}$ ，使入射角趋于零时光高与入射余弦同步衰减，从而消除该伪影 (artifact)。

### 1.3 交互相机的正交化与运动

设相机中心为  $\mathbf{c}$ ，单位视线方向为  $\hat{\mathbf{d}}$ ，固定参考竖直方向为  $\mathbf{u}_0$ （未必与当前视线正交）。当前帧的屏幕竖直方向 (screen up) 取

$$\mathbf{u}_s = \text{normalize}(\mathbf{u}_0 - (\mathbf{u}_0 \cdot \hat{\mathbf{d}})\hat{\mathbf{d}}),$$

即  $\mathbf{u}_0$  在垂直于  $\hat{\mathbf{d}}$  的平面上的投影，使竖直轴不随视线倾斜而漂移。水平轴为  $\hat{\mathbf{h}} = \hat{\mathbf{d}} \times \mathbf{u}_s$ 。

绕场景中心旋转时，先绕  $\mathbf{u}_0$  施加方位角变化，再绕  $\hat{\mathbf{h}}$  施加俯仰角变化，并对俯仰角限幅，避免视线与  $\mathbf{u}_0$  趋于共线时的万向节翻转。平移 (truck) 在  $\hat{\mathbf{h}}$  与  $\mathbf{u}_s$  张成的平面内移动  $\mathbf{c}$ ；推拉 (dolly) 沿  $\hat{\mathbf{d}}$  移动  $\mathbf{c}$ ，改变物距而视线方向不变。

## 1.4 OpenGL 预览与层次变换

OpenGL 实时预览依托图形硬件渲染管线：投影矩阵将视锥或正交棱柱映射到规范化设备坐标，模型视图矩阵将世界坐标变换到相机坐标系。透视投影由半视场角  $\theta$  与宽高比决定视锥形状；正交投影由成像平面尺寸控制可见范围。固定功能管线根据顶点法线与材质参数（漫反射、环境、高光颜色及光泽度）计算局部光照；光线追踪按同一 Blinn-Torrance 公式逐像素着色时，在方向光、无阴影条件下二者明暗分布相近，便于调整构图后再做高质量渲染。

层次变换通过矩阵栈完成：对子物体施加  $4 \times 4$  齐次矩阵  $M$  时，先将当前矩阵压栈，再左乘  $M$ ，绘制完毕后弹栈，使变换只作用于当前子树。

## 1.5 球面细分与平面近似

球心为  $\mathbf{c}$ 、半径为  $r$  的球面用经纬参数  $(\theta, \phi)$  表示，其中  $\theta \in [0, 2\pi)$ ， $\phi \in [-\pi/2, \pi/2]$ ：

$$\mathbf{p}(\theta, \phi) = \mathbf{c} + r(\cos \phi \cos \theta, \sin \phi, \cos \phi \sin \theta).$$

在  $\theta$ 、 $\phi$  方向均匀划分步长后，每个网格单元绘制为四边形。平面着色 (flat shading) 为每个多边形指定单一面法线，片元颜色在多边形内恒定。Gouraud 着色 (Gouraud shading) 在每个顶点赋予球面真法线  $\mathbf{n} = \text{normalize}(\mathbf{p} - \mathbf{c})$ ，硬件对顶点着色结果做双线性插值，相邻单元间亮度过渡更平滑；该插值作用于颜色而非法线，故仍弱于逐像素 Phong 插值。

无限平面  $\mathbf{x} \cdot \hat{\mathbf{n}} = d$  无有限边界。预览时取平面上距原点最近点  $\mathbf{o} = d\hat{\mathbf{n}}$ ，在平面内构造正交基

$$\mathbf{b}_1 = \text{normalize}(\mathbf{v} \times \hat{\mathbf{n}}), \quad \mathbf{b}_2 = \hat{\mathbf{n}} \times \mathbf{b}_1,$$

其中  $\mathbf{v}$  为与  $\hat{\mathbf{n}}$  不共线的辅助向量，再以  $\mathbf{o} \pm B\mathbf{b}_1 \pm B\mathbf{b}_2$  围成边长  $2B$  的矩形； $B$  取有限大数，以免数值溢出。

# 2 程序设计与实现

## 2.1 总体架构

系统由线性代数库、图像库、场景配置解析、几何图元、相机、材质与 OpenGL 画布组成。主程序解析命令行后读入场景；若启用图形界面，则初始化窗口并进入事件循环，按键触发光线追踪回调；否则直接执行逐像素渲染。

```
if (globalArgs.use_gui) {  
    glutInit(&argc, argv);
```

```
GLCanvas canvas;
canvas.initialize(globalParser, renderScene);
}
renderScene();
```

三维物体在求交之外提供绘制接口；材质提供局部着色与 OpenGL 材质设置接口：

```
class Object3D {
    virtual bool intersect(const Ray &r, Hit &h, float tmin) = 0;
    virtual void paint(void) const = 0;
};

class Material {
    virtual Vec3f Shade(const Ray &ray, const Hit &hit,
                        const Vec3f &dirToLight,
                        const Vec3f &lightColor) const = 0;
    virtual void glSetMaterial(void) const = 0;
};
```

球体细分步数、Gouraud 开关与掠射角高光修复由命令行设定，供预览与光线追踪共用。

## 2.2 OpenGL 预览流程

画布初始化时配置双缓冲、深度测试与方向光，注册显示、窗口缩放、鼠标与键盘回调后进入主循环。每帧清屏后根据当前相机设置模型视图矩阵，遍历光源并调用场景根节点绘制；默认单遍绘制，编译期启用高光修复时改为三遍混合，以抑制 OpenGL 端掠射角伪影。窗口尺寸变化时重新设置正交或透视投影。

## 2.3 交互相机

屏幕竖直方向由参考竖直方向剔除视线分量后得到；水平轴由视线与屏幕竖直方向叉积并归一化：

```
static Vec3f getScreenUp(const Vec3f &direction, const Vec3f &up) {
    Vec3f screenUp = up;
    screenUp -= direction * screenUp.Dot3(direction);
    screenUp.Normalize();
    return screenUp;
}
```

旋转时依次绕参考竖直轴与水平轴变换相机中心与视线，并对俯仰角限幅；平移沿水平轴与屏幕竖直方向移动中心；推拉沿视线移动中心。正交相机按窗口宽高比设置平行投影范围；透视相机按垂直视角设置透视投影；二者均根据中心、视线与参考竖直方向放置观察坐标系。

## 2.4 Phong 材质与逐像素着色

Phong 材质存储漫反射色、高光色与高光指数。局部着色先计算  $\mathbf{N} \cdot \mathbf{L}$ ；为非正时返回零，否则累加漫反射与 Blinn-Torrance 高光；启用掠射角修复时将高光再乘以  $\mathbf{N} \cdot \mathbf{L}$ ：

```
float nDotL = normal.Dot3(dirToLight);
if (nDotL <= 0.0f)
    return Vec3f(0, 0, 0);
Vec3f diffuse = componentMultiply(lightColor, diffuseColor) * nDotL;

Vec3f viewDir = ray.getDirection() * (-1.0f);
viewDir.Normalize();
Vec3f halfVector = dirToLight + viewDir;
halfVector.Normalize();
float nDotH = normal.Dot3(halfVector);
float spec = powf(nDotH, exponent);
if (specular_fix)
    spec *= nDotL;
Vec3f specular = componentMultiply(lightColor, specularColor) * spec;
return diffuse + specular;
```

OpenGL 材质设置将上述参数写入固定管线，光泽度截断至  $[0, 128]$ 。逐像素着色先叠加环境光，再对每个光源查询入射方向与颜色，处理背面法线后调用局部着色并累加：

```
Vec3f color = componentMultiply(parser.getAmbientLight(), objectColor);
for (int i = 0; i < parser.getNumLights(); i++) {
    parser.getLight(i)->getIllumination(
        hit.getIntersectionPoint(), lightDir, lightColor, distanceToLight);
    color += hit.getMaterial()->Shade(ray, shadedHit, lightDir, lightColor);
}
```

## 2.5 图元绘制

物体组遍历子物体并递归绘制。三角形在设置材质后提交三顶点及面法线。平面在切平面内构造大矩形四顶点并绘制。球体按经度、纬度步长双重循环生成四边形网格；默认每片共用面法线，启用 Gouraud 时为四顶点分别设置球面真法线：

```
float dTheta = 2.0f * (float)M_PI / thetaSteps;
float dPhi = (float)M_PI / phiSteps;
glBegin(GL_QUADS);
for (int iPhi = 0; iPhi < phiSteps; iPhi++) {
    for (int iTheta = 0; iTheta < thetaSteps; iTheta++) {
        // 由 (theta, phi) 计算四顶点; flat 或 Gouraud 设置法线后提交顶点
    }
}
```

```
}  
glEnd();
```

变换节点压栈、左乘物体到世界矩阵、绘制子物体后弹栈，与求交时用  $M^{-1}$  将射线变到局部空间相对应：

```
void Transform::paint(void) const {  
    glPushMatrix();  
    GLfloat *glMatrix = matrix.glGet();  
    glMultMatrixf(glMatrix);  
    delete[] glMatrix;  
    object->paint();  
    glPopMatrix();  
}
```

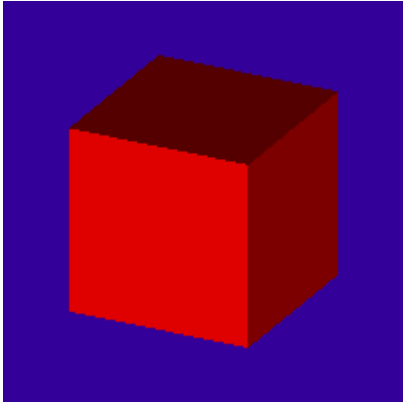
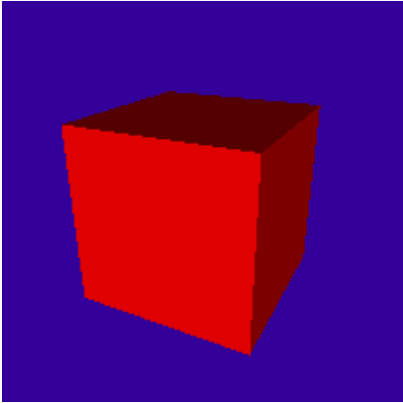
### 3 实验结果

测试命令形如：

```
# -input 输入场景； -size 图像宽高； -output 输出路径  
raytracer -input <正交立方体场景> -size 200 200 -output output3_01.tga  
# -tessellation 经度/纬度细分； -gouraud 启用 Gouraud 着色  
raytracer -input <球体场景> -size 200 200 -output output3_08.tga -tessellation 10 5 -gouraud  
# -specular_fix 光线追踪端掠射角高光修复  
raytracer -input <高光指数变化场景> -size 300 300 -output output3_09.tga -tessellation 100 50 -g
```

十二组主场景均完成光线追踪渲染；球体细分、Gouraud 与高光修复等变体另行生成。

3.1 正交与透视立方体

| 正交投影                                                                              | 透视投影                                                                               |
|-----------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
|  |  |
| 平行射线，各面明暗由法线与光源夹角决定                                                               | 近大远小；高光出现在光源与视线夹角较小的棱边附近                                                           |

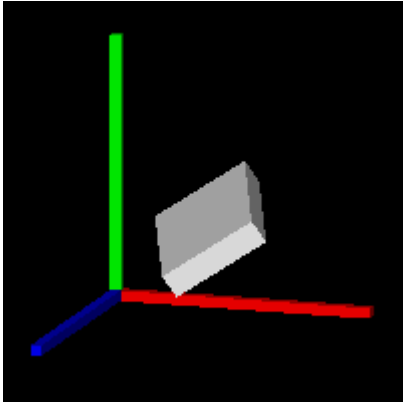
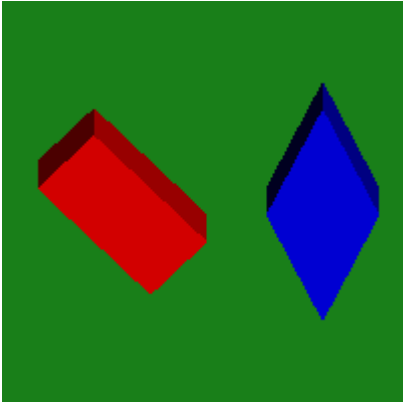
正交下明暗主要由 Lambert 项主导；透视下几何投影变化明显，Phong 高光位置随视点改变。

3.2 三角网格

| bunny (约 200 面)                                                                     | bunny (约 1k 面)                                                                       |
|-------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
|  |  |

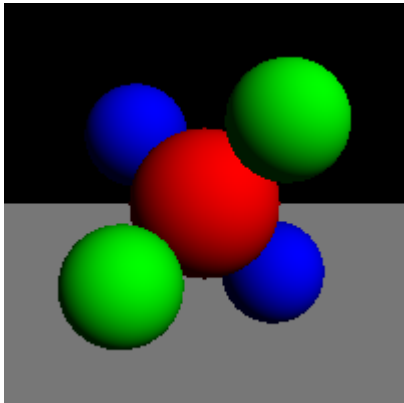
面数增加后轮廓更光滑，曲面弯曲处的高光分布更细腻。

3.3 坐标轴立方体与嵌套变换

| 多色坐标轴立方体                                                                          | 嵌套仿射变换                                                                             |
|-----------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
|  |  |

变换后几何形状与 Phong 明暗分布和预期几何相符，说明变换节点下法线与材质传递正确。

3.4 平面

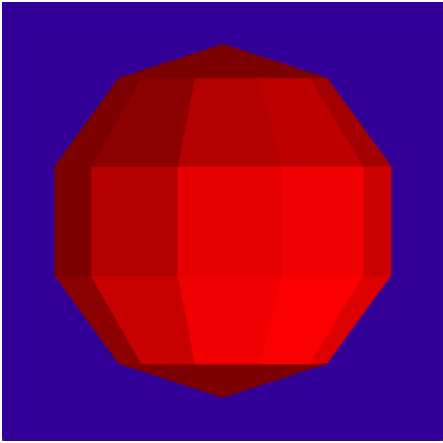
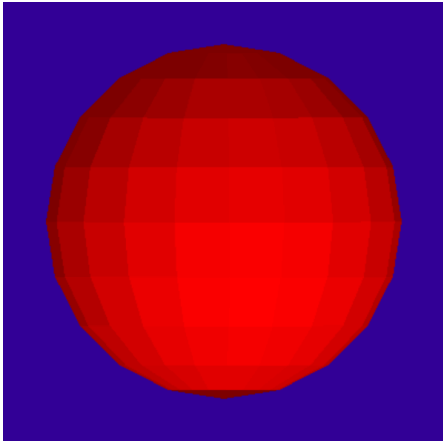
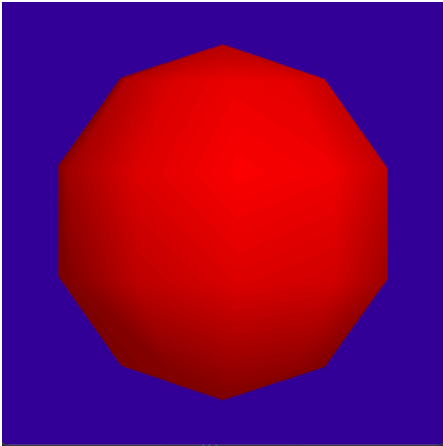
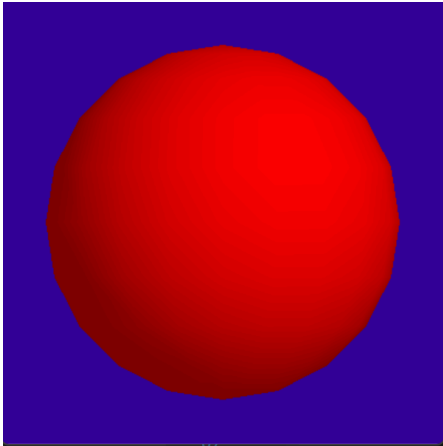


output3\_07

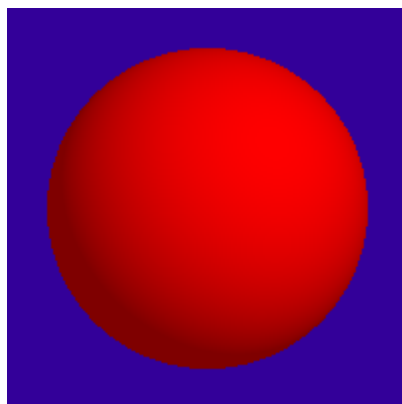
图：红、绿、蓝三平面与灰色地面在透视下相交；无限平面由大矩形近似，覆盖可见区域。

3.5 球体细分与着色模式（OpenGL 预览）

同一球体场景下，OpenGL Viewer 中经纬细分与着色模式对轮廓与明暗过渡的影响如下。

| 细分 (10, 5), 平面着色                                                                   | 细分 (20, 10), 平面着色                                                                   |
|------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
|   |   |
| 低细分, 面片棱角明显                                                                        | 提高细分后轮廓更圆, 仍可见面片边界                                                                  |
| 细分 (10, 5), Gouraud                                                                | 细分 (20, 10), Gouraud                                                                |
|  |  |
| 同低细分下亮度过渡更平滑                                                                       | 高细分 + 顶点法线插值, 视觉上最接近光滑球面                                                            |

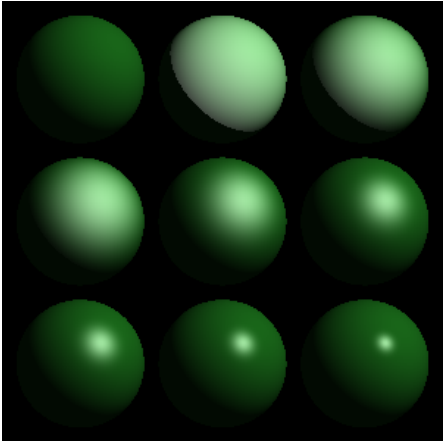
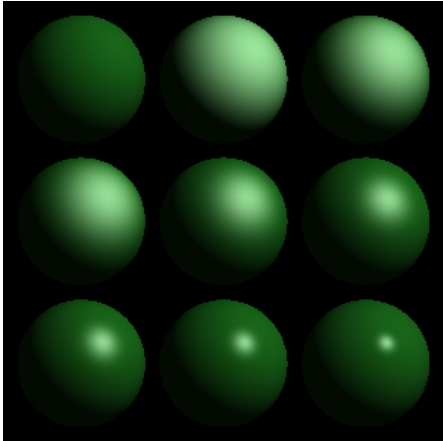
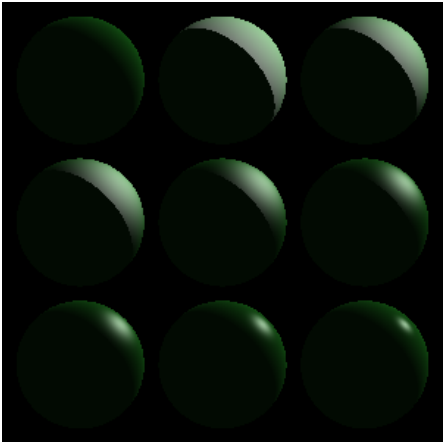
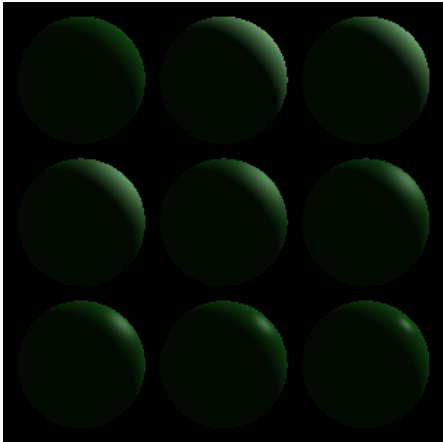
平面着色下各四边形共用面法线, 多边形棱清晰; 启用 Gouraud 后每顶点使用球面真法线, 片元颜色经双线性插值, 相邻单元间明暗过渡更连续。提高经纬步数可细化轮廓折线; 光线追踪按解析球面求交, 不受细分参数影响, 输出为光滑球面:





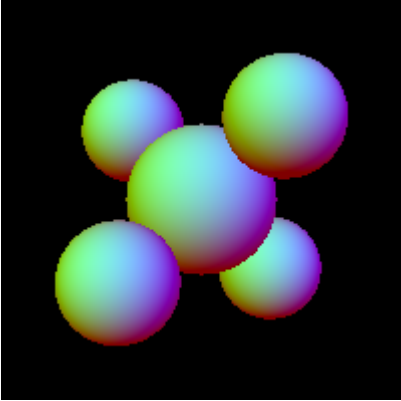
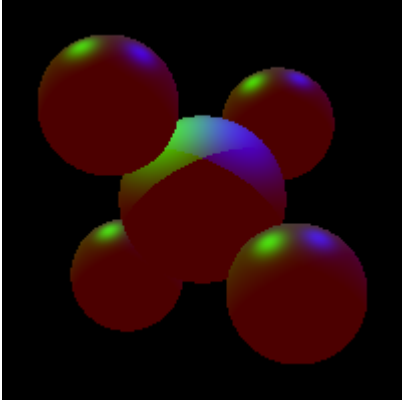
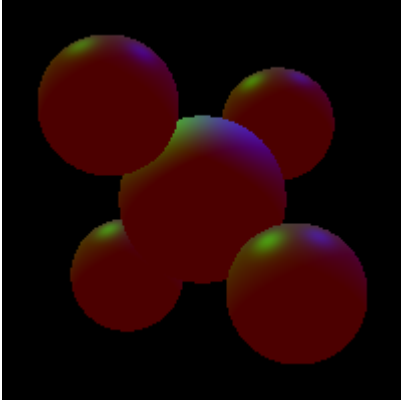
光线追踪结果（同一球体场景）。

3.6 高光指数与掠射角修复

| 正面光照                                                                               | 正面光照 + 掠射角修复                                                                        |
|------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
|   |   |
| 背光侧                                                                                | 背光侧 + 掠射角修复                                                                         |
|  |  |

九枚球体对应不同高光指数：指数越大，高光斑越小、越集中。背光侧在宽高光瓣下轮廓附近易出现伪影；乘以  $\mathbf{N} \cdot \mathbf{L}$  后，接近轮廓的高光被抑制，与正面光照场景的视觉效果更接近。

3.7 非常规光照颜色

| 漫反射色调分离                                                                           | 高光色调分离                                                                            | 高光分离 + 掠射角修复                                                                        |
|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
|  |  |  |

漫反射与高光颜色分离指定时，Lambert 项与 Blinn-Torrance 项分别携带不同色调。宽高光瓣下未修复时轮廓出现亮边；启用修复后高光集中于受光侧。

4 问题记录

编译时，在已包含标准库 iostream 并引入命名空间后直接包含 OpenGL 头文件，会触发 Windows 平台上 std::byte 与 RPC 头文件中 byte 的歧义。将 windows.h 提前包含并统一经独立头文件引入 gl / glu / glut，同时去掉对 using namespace std 的依赖后，歧义消除，问题解决。另外老问题依旧存在-光源与材质数组仍写成旧式 new (Light\*)[n] / new (Material\*)[n]，在当前 MinGW 的 g++ 下无法通过编译，改为 new Light\*[n] / new Material\*[n] 后解决。

含 TriangleMesh 的场景在根目录运行时失败，原因是其依赖 cube.obj 等网格文件。改为在 TriangleMeshes 目录下启动可执行文件后解决。

交互相机初版在旋转、平移后未及时重算水平轴，且 generateRay 直接使用未正交化的参考竖直方向，成像平面与 gluLookAt 所用坐标系不一致，预览与按键光线追踪结果错位。将原始竖直方向保留为旋转参考，用剔除视线分量后的屏幕竖直方向构造水平轴，并在每次相机运动后更新该正交基后，使得预览与光线追踪视角保持相同。

绘制变换子树时忘记 glPopMatrix，后续图元继承错误矩阵；修复为压栈、左乘、绘制、弹栈的顺序后，嵌套变换场景显示正常，问题解决。

5 总结

在既有射线求交框架上，本实验以 Blinn-Torrance Phong 局部光照扩展逐像素着色，并以 OpenGL 固定管线实现可交互的场景预览。矩阵栈处理层次变换；经纬参数化与平面着色 / Gouraud 两种法线策略逼近球面与平面；交互相机在正交化屏幕竖直方向的基础上支持旋转、平移与推拉。光线追踪端的高光修复与 OpenGL 端的三遍混合分别缓解掠射角伪影，使预览构图与最终渲染在同一光照模型下相互印证。

## 附录 (TODO list 记录)

- ☒ 为光线追踪器添加 OpenGL 预可视化界面，解析启动选项并在主程序中初始化画布、注册渲染回调
- ☒ 扩展正交相机与透视相机的投影初始化、放置及旋转 / 平移 / 推拉交互
- ☒ 由固定参考竖直方向计算屏幕竖直方向，相机运动后重算视线与水平轴并限制俯仰角
- ☒ 实现 Phong 材质抽象接口，按 Blinn-Torrance 模型计算漫反射与高光，并在场景解析中读取材质参数
- ☒ 逐像素累加环境光与各光源局部着色；方向光忽略距离衰减
- ☒ 配置 OpenGL 材质参数；光线追踪端可选将高光乘以  $\mathbf{N} \cdot \mathbf{L}$ ；OpenGL 端集成三遍高光修复
- ☒ 为物体组、三角形、平面、球体与变换节点实现 OpenGL 绘制，绘制前设置材质
- ☒ 球体按经纬细分绘制四边形网格，支持平面着色与 Gouraud 着色及命令行细分步数